Next Watch-ASE Project Report

André Rodrigues–2021222078
Bruno Almeida–2021237081
Bruno Silva–2021232021
João Samuel Diogo–2021219898
Diogo Honório–2021232043
Rodrigo Borralho–2021233455

Conteúdo

Introduction **3**

Requirement Gathering Task **4**
Traditional approach — Approx 5H . . . 4
AI-Assisted Approach — Approx 3H . . . 4
Discussion: requirement gathering Traditional vs AI . . . 5
Time and Complexity . . . 5
Quality . . . 5
Conclusion . . . 5

Design **6**
CI/CD . . . 6
Traditional Approach — Approx 6H . . . 6
AI CI/CD: Three Workflows — Approx 2H . . . 6

Frontend implementation **8**
Traditional Approach — Approx 25H . . . 8
AI Frontend implementation — Approx 3H . . . 8

Backend Implementation **10**
Traditional Approach — Approx 30H . . . 10
Overview and responsibilities . . . 10
Technologies . . . 10
Data model (Prisma) . . . 10
Server actions and auxiliary endpoint . . . 10
Authentication and security . . . 11
Time and Complexity . . . 11
Quality . . . 11
Final Consideration . . . 11
AI Backend Implementation — Approx 4H . . . 11
Overview of the Approach . . . 11
AI-Assisted Code Review and Improvement . . . 12
Debugging With AI as the Core Activity . . . 12

Feature Refinements and Adjustments 12
Linting Issues and CI Integration Challenges 12
Outcome and Efficiency 12

Testing — Approx 2H **14**

Conclusion **15**

Introduction

The rapid evolution of artificial intelligence has begun reshaping the way software systems are planned, designed, and developed. As AI-powered tools become increasingly accessible, development teams are faced with an important question: *How does an AI-assisted workflow compare to a traditional, human-driven approach?*

This project aims to explore that question by implementing the same system twice—first using conventional software engineering practices, and then recreating it with substantial support from AI tools.

The goal of this comparison is not only to evaluate differences in development time, complexity, and workflow, but also to assess the impact of AI on the quality of requirements, design decisions, code implementation, testing, and continuous integration. By contrasting both approaches across each phase of the software development lifecycle, this report highlights the strengths, limitations, and practical implications of integrating AI into real-world projects.

Ultimately, this study provides insight into how AI can augment developer productivity, streamline processes, and influence the overall development experience, while also acknowledging the areas where human judgment and manual work remain essential.

Requirement Gathering Task

The following section describes the Traditional and AI-Assisted approaches taken when gathering requirements for the application.

Each task name is highlighted followed by the respective spent hours of development.

Traditional approach — Approx 5H

Stakeholders — 20min: Team gathered together and discussed the principal stakeholders.
Existing Solutions — 20min: Team searched for existing solutions to be aware of the principal functionalities to be implemented.
Assignment Distribution — 20min: It was assigned a high-level functionality to be developed for each member (Requirement & User Stories).
Peer Review — 40min: For each member was also assigned a different functionality to review.
Standardization — 10min: Since requirements and user stories were being done with different formats, we standardized them:

- **Requirement Template:** "As a [user type/personal], I want to [functionality/feature], so that [benefit/value]"
- **Scenario Template:** "Given [context(state)], When [action/trigger], Then [expected outcome], And [additional conditions]"

Assignment Distribution 2 — 180min:

- One member defined the non-functional requirements — 60min
- One member defined constraints and expectations — 60min
- Two members conducted the global quality reviews — 60min

AI-Assisted Approach — Approx 3H

Prompt & LLM — 60min:

- Creating a set of prompts to use
- Search for which LLM to use
- Comparing results of each pair LLM & PROMPT

Peer Review — 150min: All members conducted a peer review on the generated requirements.

Discussion: requirement gathering Traditional vs AI

Time and Complexity

Time consumption was notably lower in the AI-assisted approach (3 h), representing almost half the time required for the traditional approach (5 h). This reduction results from the immediate, structured responses generated by the LLM, the decrease in manual tasks, and the overall simpler organization of the AI-assisted workflow compared to the constant coordination and restructuring required in the traditional approach.

Quality

The AI Approach proved superior in writing professionally. It generated well written User Stories, but it suffered from over-generation. It produced too many requirements and information/explanations, covering not only essential requirements/scenarios. This is a proof that AI fully relies on the prompt and has no additional judgement over output.

In the Traditional Approach, the team was better at Scoping. We fastly understood what we wanted to do and focused only on the essential features, but, manual approach was prone to Human Error. On peer-reviews many document sentences had to be re-written. The documents contained inconsistencies in formatting, so we needed to create a standard format, and still needed to correct some when doing peer-reviews.

Conclusion

The AI has proven to be superior, delivering better written requirements and scenarios, but it generates too much information, which can be an advantage or disadvantage, depending on the team goal. In our case too much information was a trade-off since more filtration was needed. Taking that in mind it still managed to take only half the time of Traditional Approach.

If we were to start again, we would prefer the Traditional Approach for gathering fundamental requirements because the discussions and research we conducted as a team helped consolidate the information in our minds. The process felt clearer and more focused, without the excess information generated by the AI-assisted method. But we would then use AI to enhance our fundamental requirements and to amplify their detail, structure, and completeness.

Design

For the design phase, both the traditional and AI-assisted versions of the system relied on AI-generated mockups. Since the team had limited experience with UI/UX design, AI tools (v0 by Vercel) were used to produce the initial visual concepts, layout suggestions, and style guidelines. However, although the AI provided a solid starting point, the generated designs did not fully align with the requirements of the system we were building. As development progressed, several adjustments and custom refinements were made manually to ensure better usability, technical feasibility, and coherence with the final product. In both approaches, the role of AI was therefore supportive rather than definitive—offering inspiration and structure, while the final decisions and modifications were driven by our own judgment and implementation requirements.

During the AI-assisted implementation, the design was also influenced by the way AI generated certain parts of the code. In several cases, the AI introduced additional interface elements—such as extra buttons or controls—to support new features it assumed would be useful.

CI/CD

Traditional Approach — Approx 6H

In the traditional approach, the team manually designed and implemented the CI/CD pipeline, explicitly defining when builds would run, which branches could trigger deployments, and how production would be protected. The pipeline was structured into three workflows: CI, PR Validation, and CD.

The CI workflow is triggered by pushes to the dev branch and is responsible for checking out the repository, installing dependencies, building the application, running static analysis and automated tests, and finally building a Docker image and pushing it to the ghc::io registry with a development tag (dev-latest). The PR Validation workflow runs on pull requests targeting main and verifies that the dev-latest image exists in the registry by pulling it, ensuring that only code that has a successfully built image can be merged. The CD workflow is triggered by pushes to the main branch after a pull request is merged; it pulls the dev-latest image and retags it as latest, promoting the previously validated artifact to a production-ready tag. The actual deployment is then triggered manually in the Render platform, which fetches and runs the image tagged as latest, adding an extra manual safeguard before changes reach the production environment.

AI CI/CD: Three Workflows — Approx 2H

The AI was explicitly instructed to build upon the already established CI, PR Validation, and CD workflows, rather than replacing them from scratch. Its goal was to automate more steps of the pipeline and simplify maintenance by refining what was already working in the traditional setup.

However, the generated workflows introduced unnecessary additional build steps and altered the pipeline in ways that were not required for the team's context. At the same time, the AI configuration omitted important environment variable settings, which led to misconfigured runtimes and ultimately contributed to application errors in production.

Instead of introducing safer, clearly separated tags such as ai-latest to distinguish AI-managed artifacts, the AI workflows continued to reuse the existing dev-latest and latest

tags for images. This lack of tag isolation meant that experimental or incorrectly built images could be promoted to production, directly impacting the live environment and making it harder to trace and roll back faulty releases.

Frontend implementation

Traditional Approach — Approx 25H

The traditional approach to the frontend implementation was initially slow, as it focused on presenting page drafts with almost every feature already in mind. This enabled a concise and purposeful development process. The user experience was straightforward: a basic home page displaying the most popular movies; a search page capable of filtering by movies, directors, and genres; functionality for creating movie collections; and a recommendation page. User interaction with the application was equally objective, allowing users to express their satisfaction by rating and commenting on movies, thereby meeting all the defined requirements. The user interface was simple yet effective, with consistent styling across pages and only minor variations.

However, there were some drawbacks, for example, the home page did not present any categories beyond trending movies and offered no filtering options, and the search page suffered from the same limitations. As a result, users had a restricted ability to explore content in more flexible or meaningful ways, which limited the overall navigability and discoverability of the application.

AI Frontend implementation — Approx 3H

In the AI-assisted frontend implementation, there were several major improvements to the application. The home page and the search page introduced detailed filtering options, providing users with more flexibility when exploring content. The movie page offered enhanced interaction features, such as the ability to share a movie with others and access more comprehensive details about it. Additionally, the collection page was upgraded to allow users to edit a collection and remove movies from it. The recommendation page also improved by presenting a match percentage and explaining the rationale behind each recommendation.

The styling across all pages remained consistent with the original design, as the AI based its improvements entirely on the existing interface rather than introducing new visual patterns. This ensured visual coherence throughout the application, though it also meant that certain aesthetic limitations of the initial version were preserved.

Overall, the AI-assisted version was significantly better in terms of the features offered to the user, providing a richer and more interactive experience. Nevertheless, it still requires minor refinements to the styling to better align the enhanced functionality with a more polished and modern visual presentation. Further improvements in layout, spacing, and component consistency could elevate the UI to the same standard as the newly added features.

In the future, we would choose to develop the frontend using the traditional approach first and then enhance it with AI assistance. However, leveraging AI to create a frontend from scratch could also significantly accelerate the development process and improve the overall delivery of this component. Taking our approach as a reference by using traditional methods in an early state, took an average of 25 hours as a whole, while using AI for improvement of our application only took around 3 hours, demonstrating that AI could accelerate the process. This hybrid strategy would allow us to retain full control over the core structure, ensuring that architectural decisions and design principles are well defined, while still benefiting from AI-driven optimizations, feature expansions, and rapid

prototyping. Ultimately, this approach would balance reliability with innovation, resulting in a more polished, efficient, and user-centric application.

Backend Implementation

Traditional Approach — Approx 30H

In this section of the report, we describe the backend implementation of our project, specifically the part developed without the assistance of Artificial Intelligence.

Overview and responsibilities

The backend was implemented manually and covers the core responsibilities of the application:

- Data persistence (Prisma + SQLite)
- Authentication (NextAuth)
- Storage of ratings and reviews (Prisma)
- Management of user collections
- Server actions
- Recommendation and search logic

The backend follows the Next.js App Router architecture. We use server actions where appropriate and small API endpoints when needed.

Technologies

- Node.js / Next.js – full-stack framework
- Prisma ORM – database modelling and access
- SQLite (development) with better-sqlite3 adapter – simple local database
- NextAuth – user authentication
- Fetch + server actions – communication between client and server
- TMDB API TS – a TypeScript-first lightweight client for The Movie Database API

Data model (Prisma)

The main models implemented in schema.prisma are the User, Movie, Rating, MovieReview, Collection, Account, Session, VerificationToken.

Server actions and auxiliary endpoint

The core backend logic was implemented using server actions, such as in actions.ts, executeActions.ts, movieActions.ts, etc. Examples of implemented functions are the signup(), the SaveRating(), the createCollection(), etc.

We've also used an auxiliary content moderation endpoint. This endpoint receives text via POST and returns a modified version, fulfilling the project requirement.

Authentication and security

- NextAuth manages the session on the backend; sensitive actions (saving rating/review/collection) require an authenticated user with valid email/name.
- Server actions verify the user by email before performing any database operation.
- Inputs are minimally validated (e.g., converting movieId to number, checking isNaN).

Time and Complexity

The backend implementation without AI assistance progressed much more slowly than initially expected. The implementation demanded extensive manual attention across all systems. Structuring the data models, defining server actions, ensuring communication between components all introduced significant complexity. Tasks such as creating CRUD operations, validating inputs, managing sessions, or adjusting Prisma queries, required repetitive and detailed work that consumed considerable time. Moreover, debugging certain issues often led to refinement that extended the development timeline. As a result, this contributed to a higher level of complexity and slowed the development pace compared to what could have been achieved with AI-assisted tools.

Quality

Although the manually implemented backend successfully delivered most of the required functionality, it presented minor inconsistencies and occasional structural irregularities. Some functions became longer than ideal, certain validations were only added later in development, and the documentation was kept to a minimal level due to time constraints. The code worked reliably, but it lacked some uniformity and was not really "clean". Elements such as naming conventions, modularity and even comments varied slightly across files, reflecting the natural evolution of the project.

Final Consideration

If the backend were to be developed again today, we would always choose to use AI Assistance. Most recent coding agents excel at generating code, accelerating repetitive tasks, and identifying mistakes that might otherwise take longer to detect. They streamline the creation of models, server actions, routes and even test suites, making the early stages of development more efficient. However, AI would not replace our manual roles. Instead, it would serve as a complementary tool speeding up the tasks while we retain full responsibility over architectural decisions, code review, validation, and testing.

AI Backend Implementation — Approx 4H

Overview of the Approach

For the AI-assisted backend implementation, we did not generate the entire codebase from scratch. Instead, we used AI tools—specifically Cursor and GitHub Copilot (free versions)—to refine, extend, and optimize it. The workflow involved providing the AI with the existing codebase and the project's requirements document, allowing it to analyze the

logic, detect inconsistencies, and propose improvements aligned with the defined specifications.

AI-Assisted Code Review and Improvement

A major part of the process involved requesting the AI to review the backend functionality we had already built. This included validating endpoints, improving database interactions, and ensuring that the project followed best practices. By referencing the requirement file, the AI was able to suggest missing validations, incomplete logic, or architectural improvements. Instead of writing the entire system autonomously, the AI acted as a collaborative reviewer, helping evolve the code into a more complete and consistent version of the intended backend.

Debugging With AI as the Core Activity

Although AI accelerated certain tasks, a significant portion of the AI-assisted development time was spent debugging AI-generated suggestions. Many of the improvements or new code blocks proposed by the tools introduced minor errors, mismatched types, or logic inconsistencies. As a result, the workflow frequently involved iterating with the AI—identifying issues, asking for corrections, testing the changes, and refining the code repeatedly. In practice, AI sped up the identification of issues but also required careful human supervision to maintain code stability.

Feature Refinements and Adjustments

The AI occasionally proposed enhancements or additional backend logic that went beyond the initial scope of the project. Importantly, it also identified and introduced features that were included in the requirements document but had not been fully implemented in the manual version. While some of these additions improved compliance with the original specification, they also required manual adjustment to integrate smoothly with the existing codebase. This collaborative approach meant combining AI-generated improvements with human decision-making, ensuring that the final backend implementation balanced efficiency, project scope, and functional accuracy.

Linting Issues and CI Integration Challenges

One of the recurring challenges during the AI-assisted development was the large number of linting errors introduced by AI-generated code. Although the logic often worked, the formatting, structure, and style frequently violated the project's linting rules. This became especially problematic when integrating the code into the CI pipeline, as linting failures prevented successful builds. To resolve this, we had to iterate repeatedly with the AI—requesting fixes, reformatting code, and validating each change—until the backend passed all automated checks. This additional effort increased the total development time and highlighted the need for consistent human validation when using AI-generated code in environments with strict CI requirements.

Outcome and Efficiency

Overall, the AI-assisted backend development significantly reduced the time spent writing new code manually, but increased the time spent reviewing, debugging, and validating

the generated solutions. Instead of replacing the development process, the AI tools acted as intelligent assistants—offering guidance, corrections, and enhancements—while still relying on human oversight to ensure correctness and alignment with the system's intended architecture.

Testing — Approx 2H

Several comprehensive tests were conducted to evaluate five critical components of our application: RecommendationsPage, CollectionsPage, CollectionsSlugPage, SearchPage, and MoviePage. All the test suites were AI-generated, resulting in 44 individual tests that comprehensively cover the core functionalities of the application. In this section, we will analyze what was tested by the AI and analyse how artificial intelligence provided significant advantages throughout the testing process.

Our AI-generated test suite employs component, integration and user interaction testing to validate all the interfaces of our application. Component testing was used to verify the rendering and visual states of all the components of the application. Integration testing was responsible for validating the API communication layers, particularly asynchronous data flow between frontend components and mocked backend services. User interaction testing covered complete workflows, such as tab switching, form validation, search input handling and button click sequence. Furthermore state transition testing covered proper handling of loading, error, empty, and populated states across all components. The tests also incorporated accessibility testing through semantic role verification and error boundary testing to guarantee graceful failure handling.

The use of AI proved to be very effective in the development of the application's test suites, it was responsible for improving the quality and efficiency of our testing process. It was possible to generate 44 tests across the five components of the application in a much faster way compared to traditional methods, this not only reduced time consumption but also implementation effort. Beyond accelerating development, AI was capable of making effective tests that covered mainly all of the core functionalities. It was possible to see that there was a phenomenal performance regarding coverage metrics, showing 96.31% statement coverage and 83.33% branch coverage, indicating that the AI-generated tests exercise nearly all code paths and decision points. As well as a positive 66.66% function coverage, that shows a balanced and strategic validation of the application's operational logic.

Conclusion

The comparative analysis of traditional versus AI-assisted software development demonstrates that AI can significantly accelerate workflows, reduce repetitive tasks, and enhance overall efficiency across multiple stages of a project. In requirement gathering, AI produced professionally written user stories and scenarios in roughly half the time of the traditional approach, though it tended to over-generate information, requiring additional filtering. For frontend and backend implementation, AI-assisted tools drastically reduced development time while enabling richer features and more complete functionality, particularly when enhancing existing work. Similarly, AI proved highly effective in generating comprehensive test suites, ensuring broad coverage and efficient validation of the system.

However, AI is not a complete replacement for human judgment. Traditional methods remain valuable for scoping, critical decision-making, and consolidating requirements, as well as ensuring architectural coherence. Additionally, AI-generated code can introduce subtle errors or inconsistencies, making debugging and integration more complex. Human oversight is therefore essential to review, correct, and validate AI suggestions, especially in environments with strict CI/CD or linting requirements.

Ultimately, a hybrid workflow—starting with human-driven design and requirements, followed by AI-assisted implementation, testing, and refinement—emerges as the most balanced approach. This strategy leverages the strengths of both human expertise and AI capabilities, resulting in faster development cycles, higher-quality code, and a more feature-rich and user-centric application.